ChatGPT

Angular Form Designer Module Architecture

Overview

This document describes the improved modular structure of the Converted Form Designer in Angular. The component allows dynamic form creation, JSON editing, and element-level property manipulation using a well-structured and scalable architecture.

Core Component: `FormDesignerComponent`

Responsibilities:

- Acts as the base shell for the form designer
- Fetches the form and module by their IDs
- Orchestrates child components
- Handles form validation by iterating over each element and invoking its `validate()` method
- Manages saving and notification logic

Child Components:

- `FormDesignComponent`
- `JsonEditorComponent`
- `ElementToolboxComponent`
- `ElementPropertiesComponent`
- `FormPropertiesComponent`
- `NotificationComponent`
- `HelpComponent`

Subcomponents

Form Design and JSON Editor

`FormDesignComponent`

- Displays form layout visually
- Iterates through rows and columns
- Contains `<app-element>` component that renders each design element by type

`JsonEditorComponent`

- Two-way editable JSON view of form design
- Validates JSON and updates the form structure accordingly

Element, Properties, and Notification Area

`ElementToolboxComponent`

- Contains draggable list of available form elements
- Grouped by element type

`ElementPropertiesComponent`

- Allows editing of selected element's properties

`FormPropertiesComponent`

- Edits global form settings like title, version, etc.

`NotificationComponent`

- Manages form notifications like onSubmit, onSave

`HelpComponent`

- Displays key editing shortcuts and help

Services and Factory

`FormService`

- Handles CRUD operations for form data
- Maintains reactive state for the current form

`ModuleService`

- Fetches module metadata
- Provides contextual constraints or templates

`NotificationService`

- Manages creation, update, and removal of form notifications

`ElementFactoryService`

- Dynamically returns instances of form elements based on their type

---

Element Classes

Base Class: `BaseElement`

```
abstract class BaseElement {
  id: string;
  label: string;
  type: string;
  abstract validate(): ValidationResult[];
}
```

Derived Classes:

- `InputElement`
- `SelectElement`
- `DateElement`
- etc.

Each class implements the `validate()` method containing logic specific to that element.

Folder Structure

```
/src/app/form-designer/
│
├── components/
│   ├── form-designer/
│   ├── form-design/
│   ├── json-editor/
│   ├── element-toolbox/
│   ├── element-properties/
│   ├── form-properties/
│   ├── notification/
│   ├── help/
│   └── shared/element/
│       └── types/
│
├── elements/
│   ├── base-element.ts
│   ├── input-element.ts
│   ├── select-element.ts
│   ├── ...
│
├── services/
```

```
│   ├── form.service.ts
│   ├── module.service.ts
│   ├── notification.service.ts
│   └── element-factory.service.ts
│
├── models/
│   ├── form.model.ts
│   ├── notification.model.ts
│   ├── enums.ts
│
├── directives/
│   └── draggable.directive.ts
│
├── utils/
│   └── form-utils.ts
│
├── form-designer.module.ts
└── index.ts
```

Module Declaration

```
@NgModule({
  declarations: [
    FormDesignerComponent,
    FormDesignComponent,
    JsonEditorComponent,
    ElementToolboxComponent,
    ElementPropertiesComponent,
    FormPropertiesComponent,
    NotificationComponent,
    HelpComponent,
    ElementComponent,
  ],
  imports: [
    CommonModule,
    FormsModule,
    ReactiveFormsModule,
    DragDropModule,
    JsonEditorModule,
  ],
  providers: [
    FormService,
    ModuleService,
    NotificationService,
```

```
    ElementFactoryService,
  ]
})
export class FormDesignerModule {}
```

This structure aims for scalability, testability, and clear separation of concerns.